DEEP RESEARCH REPORT

Algorithmic and AI Trading Systems

A multi-source survey of production trading engines, machine-learning methods, validation science, execution algorithms, and risk discipline — synthesized into a phased upgrade blueprint for the ai-trading-lab-data-engine.

AI Trading Lab

Data Engine Project — Owner Research Series

OCTOBER 2026 · VERSION 1.0

Table of Contents

1. Executive Summary 2

2. Research Scope and Method 3

3. The Open-Source Engine Landscape 3

4.1 The Nautilus kernel: parity as architecture . . . 4

4.2 OMS/EMS separation and where risk sits . . . 5

5.1 The model families and their honest trade-offs . . . 6

5.2 Adaptive ML: the FreqAI pattern . . . 6

5.3 LLM agents: real capabilities, hard limits . . . 7

6.1 The problem in one number . . . 7

6.2 The defence toolkit . . . 8

7. Strategy Families and What Survives Costs 10

8.1 Order slicing algorithms . . . 10

8.2 The square-root impact law and slippage estimation . . . 10

8.3 Transaction cost analysis . . . 11

9. Risk Discipline and Position Sizing 11

9.1 The prop-firm rulebook as a ready-made risk spec . . . 12

10. Market Data Strategy 13

11. Synthesis: Upgrade Blueprint for the Data Engine 14

12. References 16

1. Executive Summary

This report presents a deep, multi-source research survey of algorithmic and AI-driven trading systems, conducted as the preparatory phase before a major upgrade of the **ai-trading-lab-data-engine** project. The research scanned the leading open-source trading engines (NautilusTrader, QuantConnect LEAN, Microsoft Qlib, Freqtrade with FreqAI, Hummingbot, VectorBT), the academic literature on machine learning for trading (FinRL, triple-barrier labelling, combinatorial purged cross-validation, the Deflated Sharpe Ratio), the institutional execution science used by large firms (TWAP, VWAP, Implementation Shortfall, Transaction Cost Analysis, the square-root market impact law), and practitioner risk frameworks (position sizing, the Kelly criterion, prop-firm drawdown rulebooks). The goal is practical: identify precisely which capabilities separate a retail-grade trading bot from an institutional-grade trading system, and map every gap onto a concrete, phased upgrade plan for the data engine.

Three findings dominate. First, the strongest open-source engines converge on one architectural principle: **research-to-live parity**, meaning the exact same execution code path runs in backtest, in paper trading, and against a real broker, with only the data and execution clients swapped at a single boundary. Second, the most reliable destroyer of trading systems is not bad code but **backtest overfitting**: statistical research shows that trying 50 parameter combinations inflates the expected best Sharpe ratio by roughly 2.28 cross-trial standard deviations even when every strategy is worthless, which is why validation science (purged cross-validation, deflated Sharpe, walk-forward) must be built into the engine rather than left to user discipline. Third, execution quality and risk discipline, not prediction accuracy, are where institutional firms earn their durable edge: order slicing algorithms, slippage measurement, hard pre-trade risk checks, and drawdown-based kill rules are standard practice, and all of them are implementable in software at zero data cost.

2.28σ
inflated Sharpe from 50 trials of pure noise (DSR research)

0.5-2%
risk per trade used by professional fund managers

15 venues
supported by NautilusTrader's parity architecture

The synthesis chapter translates these findings into an upgrade blueprint that deliberately reuses the ai-trading-lab-data-engine's existing strengths: a 3,000-test governance-heavy codebase, a dormant rule-based strategy generator, a 30-module prediction stack, a seven-adapter broker layer, and an existing activation gate that already enforces the paper-before-live discipline. The recommended sequence is: (v1.2) auto-scan strategy discovery over an expanded indicator library; (v1.3) backtest reality modelling with fee, slippage and fill simulation plus walk-forward validation; (v1.4) execution algorithms and transaction cost analytics; (v1.5) always-on paper autopilot with an approval inbox; and (v2.0) owner-attested live trading under hard risk limits, with backtest/live code-path parity as the guiding architectural rule.

2. Research Scope and Method

The research was executed as a structured scan across three layers. The first layer is **open-source engineering**: production repositories that represent the current state of the art in publicly available trading infrastructure. The second layer is **quantitative research**: peer-reviewed work and widely cited practitioner literature on machine learning for markets, backtest statistics, and market microstructure. The third layer is **operational practice**: how institutional desks and proprietary trading firms actually configure risk, measure execution costs, and govern automated systems day to day.

Sources were gathered through more than twenty targeted web queries across engineering blogs, official documentation, arXiv papers, and practitioner publications between October 27 and October 28, 2026. Key primary sources include the NautilusTrader engineering blog and documentation, the FinRL paper and library (NeurIPS 2020 workshop paper, continuously maintained), the Freqtrade FreqAI configuration reference, López de Prado and Bailey's deflated Sharpe ratio research, published studies of the square-root impact law on the Tokyo Stock Exchange and option markets, institutional glossaries on OMS/EMS separation and transaction cost analysis, and prop-firm rule documentation from evaluation-platform vendors. Where a claim in this report depends on a specific source, the source is listed in the References chapter.

Two honesty rules were applied throughout. First, **no profit guarantees**: nothing in the surveyed literature supports the claim that any automated system reliably produces positive returns, and this report treats strategy performance as an empirical question that only paper trading and small-size live testing can answer. Second, **no fabricated numbers**: every quantitative claim is either taken directly from a cited source or is simple arithmetic (for example, compounding a 2% loss five times) that the reader can verify with a calculator.

3. The Open-Source Engine Landscape

The open-source ecosystem contains several mature engines, each with a distinct philosophy. Understanding what each one optimises for clarifies which ideas are worth borrowing into the data engine. NautilusTrader is a Rust-native, event-driven engine whose defining property is a shared kernel: the same message bus, risk engine, execution engine, portfolio state, and cache run in both backtest and live modes, with a single substitution boundary where the backtest client routes orders to a SimulatedExchange with configurable fill, fee and latency models, and the live client routes to real venue adapters for fifteen venues. QuantConnect's LEAN engine is a mature multi-asset platform reported to power more than 300 hedge funds, with detailed reality modelling of slippage, fees and fills. Microsoft's Qlib is an AI-oriented quant research platform that automates the full workflow from data through feature engineering, model training, backtest and evaluation via a single command. Freqtrade is a crypto trading bot whose FreqAI extension pioneered adaptive, periodically retrained machine learning for practitioners, and Hummingbot focuses on high-connectivity market making across more than 35 centralized and 8 decentralized exchanges.

Engine	Core strength	Signature capability	Lesson for our engine
NautilusTrader	Rust core; deterministic event-driven kernel	Research-to-live parity: identical code path in backtest and live; FillModel / FeeModel / LatencyModel at one boundary	Adopt parity as an architectural rule, not a convention
QuantConnect LEAN	Multi-asset, institutional adoption (300+ funds)	Reality modelling: slippage, fees, fill simulation inside backtests	Backtests must model costs before results can be trusted
Microsoft Qlib	End-to-end AI quant workflow	One-command pipeline: data, features, model zoo, backtest, evaluation	Automate the whole research loop, not isolated pieces
Freqtrade + FreqAI	Practitioner ML bot	Adaptive retraining on a schedule; feature expansion across timeframes and correlation pairs	Models must retrain as markets drift; staleness is a bug
Hummingbot	Market making and connectivity	35+ CEX and 8 DEX connectors behind one adapter interface	A clean broker adapter contract enables breadth cheaply
VectorBT	Vectorised research speed	NumPy-vectorised parameter sweeps across thousands of configurations	Auto-scan needs vectorised loops to stay interactive

Table 1. Open-source trading engines compared

Two patterns cut across all six engines. The first is that none of them treats a strategy as a hand-written one-off script: every serious platform has a structured strategy object with lifecycle hooks, parameters, and declared data requirements, which is what makes automated scanning and optimisation possible. The second is that all of them separate the trading kernel from connectivity: venues, brokers and data providers are adapters behind an interface. The data engine already follows the second pattern with its seven broker adapters and BrokerAdapter contract; the first pattern is partially present in the lab module and should be strengthened during the auto-scan upgrade.

4.1 The Nautilus kernel: parity as architecture

NautilusTrader's central design document describes a conviction that took a decade of engineering: research and live trading belong in one system. Its NautilusKernel is the core of both the BacktestEngine and the LiveNode, sharing messaging, data handling, risk, execution, portfolio state and caching across both environments. A user writes one Strategy class with event handlers such as on_bar or on_order_filled, and that exact component runs unchanged against historical data or against a live venue. Determinism is enforced structurally: the kernel is single-threaded by design, events carry dual nanosecond-precision timestamps (ts_event for domain time, ts_init for system creation time), and backtest clocks advance time explicitly so that the same data through the same strategy produces the same event sequence every run. The engineering payoff is profound: a backtest result is reproducible, an audit trail is meaningful, and a strategy that worked in simulation cannot behave differently in production because of hidden code-path differences.

The substitution boundary deserves special attention because it is the exact place where realism lives. In backtest, orders route to a SimulatedExchange with configurable fill models, fee structures and latency models; in live, they route to real venue adapters. This means the engine's realism budget is concentrated in one place: improving fill logic or slippage assumptions upgrades every strategy simultaneously. The ai-trading-lab-data-engine currently has separate backtest code (the lab backtester) and paper execution code (the paper simulator), which violates parity and is the single most consequential architectural gap identified by this research. The v1.3 upgrade should converge both paths onto one execution engine with pluggable cost models, exactly as Nautilus does.

4.2 OMS/EMS separation and where risk sits

Institutional platforms separate the Order Management System, which owns the lifecycle of orders from intention to settlement, from the Execution Management System, which owns the efficient interaction with liquidity venues. The distinction matters because each layer fails differently: an OMS failure corrupts records and positions, while an EMS failure wastes money through poor fills. Both layers sit on top of a pre-trade risk gate that every order must pass before leaving the system: notional caps, position limits, fat-finger price-band checks, duplicate-order suppression, and a kill switch that can flatten or cancel everything instantly. NautilusTrader formalises this as a dedicated RiskEngine through which every order flows, and its practitioners highlight a related operational safeguard, the deadman's switch: a server-side cancel-all timer that the engine continuously refreshes, so that if the process dies or loses connectivity, the venue cancels its open orders automatically instead of leaving them exposed.

Production component	Role	Data engine status
Trading kernel / message bus	Single ordered event stream; deterministic replay	Present (event-driven runtime, flock single-writer state)
Data engine	Normalised market data routing, bar aggregation	Present (ingestion, derivatives, feed composition)
Execution engine (shared)	One code path backtest = paper = live	Gap: lab backtester and paper simulator are separate; converge in v1.3
Simulated exchange with fill/fee/latency models	Realistic fills inside backtests	Gap: lab backtest is cost-free; add cost models in v1.3
Risk engine with pre-trade checks	Notional, position, price-band, duplicate checks	Partial: risk engine and portfolio modules exist; wire to every order path
Kill switch / deadman switch	Instant halt; auto-cancel on connectivity loss	Partial: durable kill switch exists; add venue-side deadman in v2.0
Audit trail with dual timestamps	Event-level provenance for every state change	Strong: provenance, evidence, experiment registry, journals

Table 2. Production architecture components mapped to the data engine

The mapping in Table 2 is encouraging: the data engine's foundations (kernel discipline, audit trail, broker abstraction, risk modules) are institutional in shape, and the gaps are concentrated, well-understood, and buildable. This is the practical meaning of the earlier finding that the project already has strong bones: the upgrade is a wiring and realism problem, not a rewrite.

5.1 The model families and their honest trade-offs

Surveying the applied literature and the major libraries, four model families dominate practical AI trading. Gradient-boosted trees (XGBoost, LightGBM) remain the workhorse for tabular financial features: they train fast on CPU, handle heterogeneous indicators natively, and are hard to beat on daily-to-hourly horizons. Recurrent networks (LSTM, GRU) capture temporal dependence and were the deep-learning default for years, with comparative studies repeatedly finding them competitive with, and sometimes superior to, early transformer variants on financial series. Transformer-family models bring long-range attention and are the current research frontier, but they are data-hungry and prone to overfitting on the relatively short, noisy series retail traders work with. Reinforcement learning trains an agent to act rather than predict: FinRL, the reference open-source library, packages DQN, DDPG, PPO, SAC, A2C and TD3 agents behind a gym-style environment with transaction costs, market liquidity and risk aversion folded into the reward, and its feature pipeline computes technical indicators (MACD, RSI, ADX, CCI) via the stockstats library.

Family	Strengths	Weaknesses	Realistic role in the engine
Gradient boosting (LightGBM/XGBoost)	Fast, tabular-native, strong baselines	No native temporal structure; needs careful feature lags	First ML zoo member; direction and volatility classification
LSTM / GRU	Temporal memory; mature tooling	Slow to train; sensitive to scaling and window choice	Second zoo member behind a feature flag
Transformers	Long-range attention; research frontier	Data-hungry; overfit risk on short series	Later addition; only with purged validation
Reinforcement learning (FinRL-style)	Learns policies, includes costs in reward	Unstable training; reward hacking; opaque	Research track only; never wired to execution without gates
LLM agents	Reasoning over news and context; hypothesis generation	Hallucination; 100 ms-10 s latency; API cost; non-determinism	Assistant and research layer; never the execution decision-maker

Table 3. ML model families in trading

5.2 Adaptive ML: the FreqAI pattern

Freqtrade's FreqAI extension is the most instructive practitioner design for making machine learning survive live markets. Its configuration declares a training window (for example 30 days), a retraining cadence (for example every 7 days), feature expansion across multiple timeframes, shifted candles, and correlation pairs, and the system then periodically retrains models as new data arrives, purging stale models so that predictions always come from a fit on recent regime data. The architectural lesson is that **model freshness is a scheduled engineering concern**, not an ad-hoc manual task: drift is assumed, and the pipeline is built around it. The data engine's prediction stack already contains drift detection, calibration, regimes and uncertainty modules; what is missing is the scheduling loop that retrains on a timer and refuses predictions from stale models, which is a tractable v2.1 feature. A key detail worth copying is FreqAI's explicit outlier and dissimilarity handling: when live data drifts too far from the training distribution, the system can be configured to refuse to predict rather than to predict confidently from an extrapolating model.

5.3 LLM agents: real capabilities, hard limits

The 2024-2026 wave of LLM trading research separates cleanly into useful and overstated roles. Useful: sentiment extraction from news with contextual understanding, trading hypothesis generation for later quantitative validation, code generation for research pipelines (with human review), and multi-step market reasoning chains. Overstated: using an LLM as the direct execution decision-maker. The surveyed literature is consistent about the limits: hallucination produces plausible but false market claims, API latency (roughly 100-500 ms round trip and up to seconds for reasoning models) rules out anything beyond low-frequency decisions, costs scale with token volume, and outputs are non-deterministic, which breaks the reproducibility that audited trading requires. A 2026 survey frames LLM trading agents as expert-system decision pipelines and identifies auditing as the field's immediate bottleneck. The FLAG-Trader architecture is a sensible middle path: the LLM generates trading hypotheses, a reinforcement-learning or quantitative layer validates and executes them, and results feed back. For the data engine this maps to keeping the ledger-backed deterministic assistant for account questions, and allowing LLM layers to propose, never to execute: proposals enter the same backtest-paper-approval pipeline as any other strategy.

> *LLMs may generate plausible-sounding but incorrect analysis: fabricated data, incorrect causal relationships, overconfident predictions. Coping strategy: always verify with real data.*

6.1 The problem in one number

Backtest overfitting is the process by which a worthless strategy is tuned until its historical record looks excellent. The mechanism is selection, not complexity: every parameter trial produces a Sharpe estimate carrying sampling error, and the researcher reports the maximum of the batch, which contains the largest positive error by construction. Bailey and Lopez de Prado (2014) give the benchmark: under a null in which every trial has zero true skill, the expected maximum Sharpe across N trials scales with the variance of trial Sharpes and grows with N through the tail of the normal distribution. The practical consequence is a deflation bar that any claimed edge must clear. Published multipliers of the cross-trial standard deviation are tabulated below; a 50-combination sweep raises the expected noise Sharpe by 2.28 standard deviations even

when every configuration is a coin flip. Bailey's sharper formulation is worth quoting to the team: not reporting the number of trials involved in identifying a successful backtest is a form of misrepresentation.

Combinations tried	Noise bar (σ of trial Sharpes)	Interpretation
4	1.05	Even a tiny sweep inflates the winner
8	1.46	Typical first optimization session
16	1.80	Auto-scan territory; reporting trials is mandatory
25	2.00	Best-of-25 must beat 2σ of the trial spread
50	2.28	The engine must surface this bar next to every leaderboard

Table 4. Expected maximum noise Sharpe, in cross-trial standard deviations (DSR research)



Figure 1. The deflated-Sharpe noise bar rises with the number of parameter trials (values from Table 4; the bar is the expected best Sharpe produced by pure noise).

For the data engine the operational translation is direct: the auto-scan feature planned for v1.2 must record the number of trials it ran, display the noise bar next to its leaderboard, and downgrade any winner whose Sharpe does not clear it. This turns an abstract statistical warning into a number the owner sees in the app, and it is cheap to implement because the engine already computes Sharpe-family metrics in its strategy metrics module.

6.2 The defence toolkit

The literature prescribes a layered defence. Walk-forward analysis rolls an expanding or sliding window through history so that every reported period was genuinely out of sample at the time. Purged k-fold and combinatorial purged cross-validation (CPCV) split the series into blocks whose overlapping labels are

removed from training, then rotate which blocks serve as the test, so a result must survive many ways of cutting the sample rather than one flattering cut. The triple-barrier labelling method from Lopez de Prado's work frames each trade with a profit barrier, a loss barrier and a time barrier, producing labels that reflect how positions actually end rather than arbitrary fixed horizons. The Probability of Backtest Overfitting (PBO) measures how often the in-sample winner degrades out of sample across combinations. Finally, a standard error taxonomy keeps the data honest: backfill bias, survivorship bias, look-ahead bias, optimistic transaction costs, ignored capacity and slippage, performance inflation from non-normal returns, and multiple testing, which the literature calls the 800-pound gorilla because it silently invalidates results that look individually significant.

Error	What it does	Engine defence
Look-ahead bias	Signal uses information not available at decision time	Signal computed on bar close, order filled next bar open; enforced in the shared execution engine
Survivorship / backfill	History only contains assets that survived	BarStore records full upload provenance; bundles document source and gaps
Optimistic costs	Fee-free, spread-free fills inflate returns	v1.3 fee and spread models on by default; costs shown on the leaderboard
Capacity / slippage	Assumes infinite liquidity at the touch	v1.4 square-root impact estimate per order; TCA report per fill
Multiple testing	Best-of-N looks skilled when N is unreported	Auto-scan logs trial count and displays the DSR noise bar
Performance inflation	Non-normal returns flatter Sharpe	Report Sortino, Calmar, max drawdown alongside Sharpe (metrics module already does)

Table 5. Backtest error taxonomy and engine defences

7. Strategy Families and What Survives Costs

Strategy research organises naturally into families, and the evidence about each is more nuanced than marketing suggests. Trend following is the most robustly documented family: systematic studies and decades of managed-futures practice support the claim that capturing sustained moves with trailing exits produces positive expectancy across many markets, with win rates often below 50 percent compensated by larger average wins. Mean reversion profits from temporary deviations toward an average; it produces high win rates and smooth equity in ranging regimes but is exposed to sharp losses when a range breaks, which is why regime detection matters. Breakout strategies trade range exits and news-driven expansion; they suffer in chop but are the natural complement to mean reversion. Market making earns the spread by providing liquidity but demands infrastructure and inventory-risk management that only makes sense with cheap, fast connectivity. Grid and martingale variants are popular in retail circles and reliably catastrophic without hard drawdown caps, because they average into losing positions with rising size; the prop-firm literature's daily-loss rules exist precisely because such behaviour ends accounts.

For the data engine's target markets (gold and major FX pairs via MT5/Exness, with crypto adapters available), the practical conclusion is that the engine should implement the four already-planned families (trend via EMA, cross via dual EMA, reversal via RSI, breakout via channel) plus ATR-based stop sizing and volatility-scaled position sizing, and treat regime detection as a first-class signal: the existing prediction stack's regime module can label trend versus range states, and the auto-scan leaderboard should report performance conditioned on regime rather than blended into one misleading average. A strategy family that only works in trending regimes is not a bad strategy; it is a conditional strategy, and the engine's job is to expose the condition.

8.1 Order slicing algorithms

Institutional execution treats the question, how do I get from decision to position, as a solved-but-nontrivial engineering problem with a small standard toolkit. TWAP slices a parent order into equal pieces at regular time intervals; it is simple, deterministic, and minimises timing risk without using volume information. VWAP slices proportionally to a historical or real-time volume curve, following liquidity instead of the clock, and is the standard benchmark for buy-side execution quality. Implementation Shortfall (IS) algorithms, rooted in the Almgren-Chriss framework, optimise the trade-off between market impact (favouring fast execution) and timing risk (favouring patient execution), often parameterised by urgency. Percentage-of-Volume (POV) participates at a fixed fraction of observed market volume. For the data engine, the relevant insight is that even a two-level implementation, a time-slicing wrapper for larger orders and a simple participation cap, captures most of the benefit at retail scale, because the average retail order on gold or a major pair is small relative to venue liquidity; the wrapper's real value is discipline (never dump a full position into one print) rather than alpha.

8.2 The square-root impact law and slippage estimation

Market microstructure research has established one of the few near-universal laws in finance: the price impact of a metaorder grows approximately with the square root of its executed size relative to daily volume. The law was first documented systematically on equity markets, confirmed on futures, and subsequently shown to hold in option markets as well, with Tokyo Stock Exchange data revealing its mechanical origin in order-book liquidity consumption; studies also identify a crossover from linear impact for very small orders to square-root behaviour at scale. For the engine this yields a cheap, defensible slippage model: estimated impact in basis points proportional to the square root of (order notional / typical daily volume), calibrated with a conservative constant, added to the spread and fee assumptions inside backtests (v1.3) and used as a pre-trade warning on unusually large orders (v1.4). It will not be exact, but it is honest, monotone, and dramatically better than the implicit assumption of zero impact that currently underlies the lab backtester.

8.3 Transaction cost analysis

TCA is the institutional practice of measuring every trade against benchmarks so that execution quality becomes a managed number instead of a feeling. The core metrics are the arrival-price shortfall (average fill versus the mid-quote at decision time), implementation shortfall decomposed into execution cost, opportunity cost of unfilled quantity, and fixed fees, and slippage against expected price per fill. Analysis is split into pre-trade (estimating expected costs before execution to choose an algorithm) and post-trade (auditing realised costs to improve the next execution). The data engine already records fills in its ledgers with timestamps and prices, which is the hard part; the v1.4 TCA upgrade is mostly arithmetic over data that already exists: capture the decision-price mid at signal time, store it alongside the fill, and compute the shortfall distribution per strategy, per session and per broker. A weekly TCA digest in the monitor app would give the owner the same feedback loop institutional desks rely on.

9. Risk Discipline and Position Sizing

The practitioner consensus on sizing is blunt: position sizing explains more of the variance in trader outcomes than strategy selection. Two traders running the identical signals with different sizing conventions will diverge dramatically, because compounding through drawdowns is mathematically asymmetric: a sequence of losses requires a disproportionately larger gain to recover, and the required recovery grows steeply with per-trade risk. Professional fund managers typically risk 0.5 to 2 percent of the account per trade; the table below shows the arithmetic for five consecutive losses at various fixed-percentage risk levels, which is a realistic losing streak even for good strategies. The Kelly criterion, Kelly percent = W - (1 - W)/R where W is win rate and R is the win/loss ratio, offers a theoretical growth optimum, but full Kelly assumes accurately known probabilities and tolerates enormous interim drawdowns, so practice uses fractional Kelly (a quarter to a half) or simple fixed-fraction rules; the criterion's real value in an engine is as a sanity signal, since a strategy whose Kelly percentage is negative has no demonstrated edge at all.

Risk per trade	Account after 5 losses	Gain required to recover
1%	-4.9%	+5.2%
2%	-9.6%	+10.6%
3%	-14.1%	+16.4%
5%	-22.6%	+29.2%
10%	-41.0%	+69.5%

Table 6. Drawdown arithmetic for five consecutive losing trades

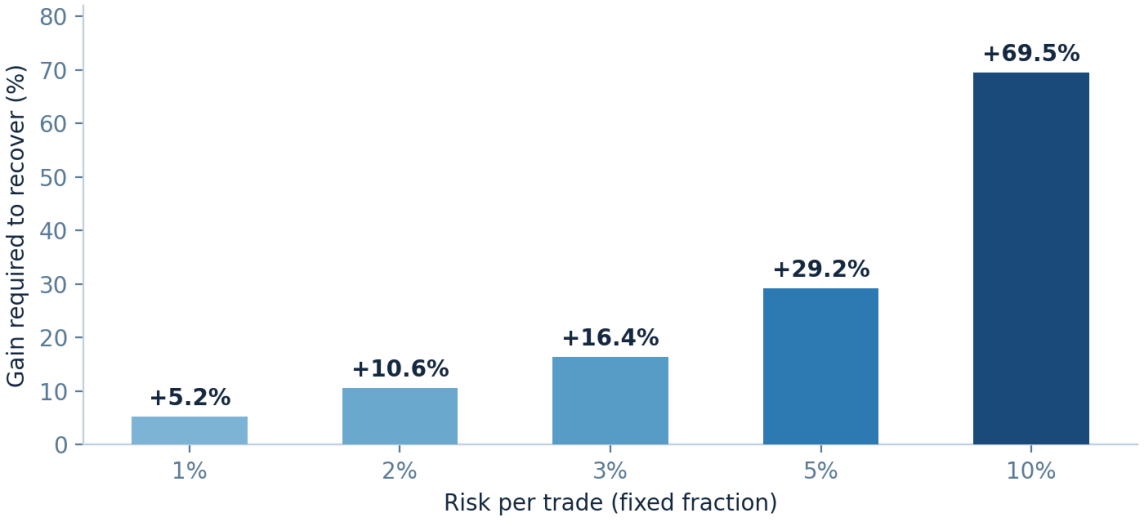


Figure 2. Recovery gain required after five consecutive losses, by fixed risk per trade (values from Table 6; compounding is asymmetric: losses hurt more than equal-sized gains help).

9.1 The prop-firm rulebook as a ready-made risk spec

Proprietary trading firm evaluation rules are a gift to engine design: they encode, in explicit numbers, the loss-discipline framework that the industry has converged on. A typical rulebook specifies a maximum daily loss (commonly around 5 percent, sometimes 4), a maximum total drawdown measured either statically from the initial balance or as a trailing floor beneath the account's high-water mark, sometimes a per-trade loss cap, minimum trading days to prevent gambling on single shots, and consistency rules that penalise profit concentration in one day. Static versus trailing drawdown matters enormously: a 100,000 account that peaks at 105,000 and falls to 94,000 is still inside a static 10 percent floor of 90,000 but has breached a trailing 10 percent floor of 94,500, because the trailing rule protects profit as well as capital. The data engine's risk module can adopt this rulebook directly as a configurable policy: daily-loss circuit breaker, high-water-mark trailing drawdown with automatic deactivation, per-trade notional and loss caps, and a mandatory cooldown after any limit breach, all enforced pre-trade by the risk engine and reported in the monitor app. This gives the owner institutional-style governance without inventing rules, and it aligns the engine with the discipline that funded traders are contractually required to demonstrate.

10. Market Data Strategy

Every downstream capability inherits the quality of its bars, so the data plan matters as much as the models. For FX and gold, the practitioner-standard free source is Dukascopy's historical data service, which provides tick-level and bar-level history that traders widely use for backtesting, with the standard caveat that intraday quality and spread assumptions vary and must be documented. MetaTrader 5 itself exposes copy_rates functions to Python through the official MetaTrader5 package, which the engine's platform layer already wraps; running against the owner's Exness demo terminal gives broker-consistent bars whose prices match the venue that would eventually be traded, eliminating a whole class of basis errors. For crypto, exchange APIs and the adapters already in the broker catalog cover OHLCV needs. The storage recommendation is Parquet (or Arrow) files partitioned by symbol and timeframe: columnar, compressed, fast to slice, and readable by every analytics tool, with the engine's existing BarStore contract preserved as the API layer above the files.

Source	Markets	Granularity	Cost	Notes
MT5 terminal (Exness demo)	FX, metals, indices	Ticks, bars from M1	Free	Broker-consistent prices; already integrated via platform layer
Dukascopy history	FX, gold CFD	Ticks, bars	Free	Practitioner standard for backtests; document spread assumptions
Exchange APIs (crypto adapters)	BTC, ETH majors	Trades, bars	Free	Matches the mock adapters already in the broker catalog
Paid feeds (Databento-class)	Futures, equities	Ticks, L2	Paid	Not required for the current scope; revisit if markets expand

Table 7. Data sources for the engine's target markets

Two quality disciplines complete the plan. First, point-in-time honesty: every bar bundle should carry its source, retrieval time, and known gaps, so that a backtest can be reproduced exactly from the same bytes; the engine's provenance and evidence modules already enforce this pattern for experiments and should extend to data. Second, quality gates before admission: duplicate detection, monotone timestamps, outlier screening, and minimum-history thresholds are all already implemented in the lab bar validation path and should become the single mandatory gateway for every data source, not just manual uploads.

11. Synthesis: Upgrade Blueprint for the Data Engine

The research converges on a phased blueprint in which each phase produces a working, testable release of the full stack (engine, Windows package, monitor app, admin app) and each phase's features map to specific findings of this survey. The guiding principles, in priority order: honesty of the backtest before cleverness of the strategy (costs and validation first); one execution code path from research to live (parity); governance gates unchanged (paper-first, explicit human approval for anything further); and owner-visible statistics (trial counts, noise bars, TCA, risk state) so that the human in the loop decides with numbers, not vibes.

Phase	Delivers	Borrows from
v1.2	Auto-scan: grid sweep over 4 strategy families x 20-indicator library with vectorised backtests; Top-5 leaderboard with trial count and DSR noise bar; one-tap send-to-paper	VectorBT (vectorised sweeps); Freqtrade hyperopt concept; DSR research (Table 4)
v1.3	Reality modelling: shared execution engine for backtest+paper (parity); fee, spread, square-root impact cost models; walk-forward validation with purged folds; triple-barrier labels	NautilusTrader parity architecture and SimulatedExchange concept; LEAN reality modelling; Lopez de Prado validation toolkit
v1.4	Execution and TCA: TWAP time-slicing wrapper; participation cap; per-fill TCA ledger (arrival-price shortfall); weekly execution digest; portfolio allocation across active strategies	Institutional EMS/TCA practice; square-root impact law studies
v1.5	Autopilot paper mode: scheduled MT5 data fetch; always-on paper evaluation of activated strategies; approval inbox with results; regime-conditioned reporting	FreqAI scheduling pattern; Qlib one-command workflow; existing prediction regime module
v2.0	Live path (owner-attested only): real broker execution adapter behind every pre-trade risk check; daily-loss circuit breaker; trailing high-water-mark drawdown deactivation; venue-side deadman cancel-all; per-trade ledger and kill switch unchanged	Prop-firm rulebooks; Nautilus RiskEngine and deadman's switch; existing governance gates
v2.1	ML zoo (optional track): LightGBM direction model first; scheduled retraining with staleness refusal; drift-gated predictions; LLM layer restricted to research proposals	FinRL feature pipeline; FreqAI retraining and dissimilarity refusal; LLM-agent survey limits

Table 8. Phased upgrade blueprint

The blueprint's sequencing is deliberate. v1.2 delivers immediate owner-visible value (the auto-scan) while introducing the statistical reporting that later phases depend on. v1.3 is the highest-leverage engineering phase because parity plus cost models upgrade every present and future strategy at once, and because walk-forward results will naturally shrink the leaderboard's optimism before anyone risks money. v1.4 and v1.5 convert research infrastructure into an operating trading desk, and v2.0's live path arrives only after the

owner has watched paper results produced by the same code that would be trading, measured execution costs, and configured hard risk limits, at which point the human approval gate stops being a formality and becomes an informed decision.

Finally, the expectation-setting note that belongs in any honest engineering document: none of these upgrades guarantees profitability, and the surveyed literature is unanimous that most retail strategies fail on costs and overfitting before they fail on signal. What the blueprint does guarantee is that every number the owner sees is real (costs modelled, trials counted, fills measured, losses capped by construction), which is precisely the property that separates professional systems from the retail majority, and the property that gives a strategy any chance of compounding.

Nautech Systems (2026). Why NautilusTrader exists. NautilusTrader engineering blog. https://nautilustrader.io/blog/why-nautilustrader-exists

Nautech Systems (2026). NautilusTrader documentation and nautilus-backtest crate reference. https://nautilustrader.io/docs/latest and https://docs.rs/nautilus-backtest

QuantConnect (n.d.). Lean Algorithmic Trading Engine. GitHub repository. https://github.com/QuantConnect/Lean

Microsoft Research (n.d.). Qlib: An AI-oriented quantitative investment platform. https://qlib.readthedocs.io/

Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., and Wang, C. D. (2020). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. arXiv:2011.09607. https://arxiv.org/abs/2011.09607

Freqtrade contributors (2023-2025). FreqAI configuration documentation. https://www.freqtrade.io/en/stable/freqai-configuration/

AI4Finance Foundation (n.d.). FinGPT repository and LLM research overview. https://github.com/AI4Finance-Foundation/FinGPT

Waylandz (2025). LLM research in quantitative trading: survey of FinGPT, BloombergGPT, FLAG-Trader and limitations. https://waylandz.com/quant-book-en/LLM-Research-in-Quantitative-Trading

Lopez-Lira, A. (2025). Can large language models trade? Evaluating LLM agents in stock markets. Paper presented at the Bank of Finland financial stability event. https://www.suomenpankki.fi/globalassets/bof/en/financial-stability/events/

Bailey, D. H., and Lopez de Prado, M. (2014). The deflated Sharpe ratio: correcting for selection bias, backtest overfitting and non-normality. Journal of Portfolio Management, 40(5). Summary tables via Saral (2026), https://saral.money/blog/backtest-overfitting-deflated-sharpe

Bailey, D. H., Borwein, J., and Lopez de Prado, M. Backtest overfitting tools. https://www.davidhbailey.com/dhbpapers/overfit-tools.pdf

Alphavo (2026). Combinatorial purged cross-validation (CPCV): method description. https://alphavo.io/en/methods/cpcv

Palomar, D. (n.d.). The dangers of backtesting. In Portfolio Optimization: Theory and Application, section 8.3. https://portfoliooptimizationbook.com/book/8.3-dangers-backtesting.html

Menaldo, S. (2026). Transaction cost analysis: implementation shortfall, arrival price, VWAP, TWAP, slippage. Medium. https://medium.com/@simomenaldo/transaction-cost-analysis-7b8f92164b8b

Quod Financial (2025-2026). EMS glossary and Transaction Cost Analysis in institutional trading. https://www.quodfinancial.com/

Capital Fund Management (2016). The square-root impact law also holds for option markets. https://www.cfm.com/the-square-root-impact-law-also-holds-for-option-markets/

Square-root law universality studies on Tokyo Stock Exchange data (2025). Physical Review Letters supplemental material and arXiv preprints on double square-root impact. https://arxiv.org/html/1802.08502v5

OptionVisualizer (2026). Position sizing and the Kelly criterion: percentage risk model tables. https://www.optionvisualizer.com/documentation/education/position-sizing

TradeCore (2026). Prop firm challenge rules: profit targets, daily loss, static vs trailing drawdown, consistency rules. https://tradecore.com/resources/blog/prop-firm-challenge-rules

Algotrade Knowledge Hub (n.d.). Evaluation of execution algorithms with TWAP and VWAP benchmarks; measuring implementation shortfall. https://hub.algotrade.vn/knowledge-hub/

Dukascopy Bank (n.d.). Historical data service: history bars and ticks. https://www.dukascopy.com/wiki/en/development/strategy-api/historical-data/

12. References